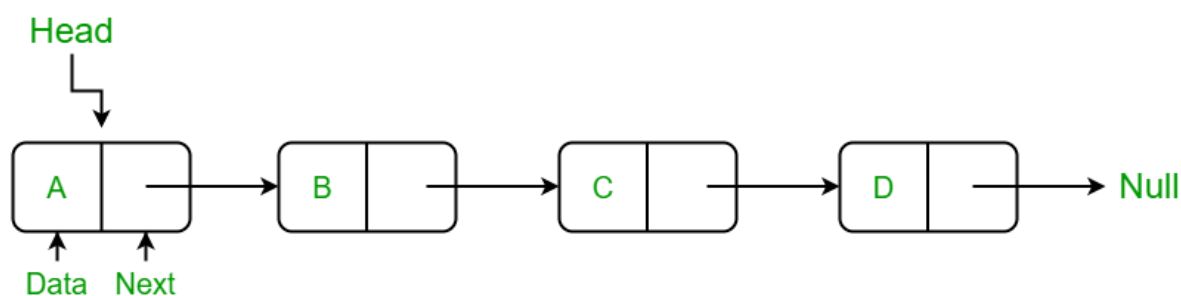


Applications of linked list data structure

Last Updated : 11 Jul, 2025

A **linked list** is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers as shown in the below image:



Linked-List

Applications of linked list in computer science:

1. Implementation of [stacks](#) and [queues](#)
2. Implementation of graphs: [Adjacency list representation of graphs](#) is the most popular which uses a linked list to store adjacent vertices.
3. Dynamic memory allocation: We use a linked list of free blocks.
4. Maintaining a directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of the linked list
7. Representing sparse matrices

Applications of linked list in the real world:

1. Image viewer - Previous and next images are linked and can be accessed by the next and previous buttons.
2. Previous and next page in a web browser - We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.

3. Music Player - Songs in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.
4. GPS navigation systems- Linked lists can be used to store and manage a list of locations and routes, allowing users to easily navigate to their desired destination.
5. Robotics- Linked lists can be used to implement control systems for robots, allowing them to navigate and interact with their environment.
6. Task Scheduling- Operating systems use linked lists to manage task scheduling, where each process waiting to be executed is represented as a node in the list.
7. Image Processing- Linked lists can be used to represent images, where each pixel is represented as a node in the list.
8. File Systems- File systems use linked lists to represent the hierarchical structure of directories, where each directory or file is represented as a node in the list.
9. Symbol Table- Compilers use linked lists to build a symbol table, which is a data structure that stores information about identifiers used in a program.
10. Undo/Redo Functionality- Many software applications implement undo/redo functionality using linked lists, where each action that can be undone is represented as a node in a doubly linked list.
11. Speech Recognition- Speech recognition software uses linked lists to represent the possible phonetic pronunciations of a word, where each possible pronunciation is represented as a node in the list.
12. Polynomial Representation- Polynomials can be represented using linked lists, where each term in the polynomial is represented as a node in the list.
13. Simulation of Physical Systems- Linked lists can be used to simulate physical systems, where each element in the list represents a discrete point in time and the state of the system at that time.

Applications of Circular Linked Lists:

1. Useful for implementation of a queue. Unlike [this](#) implementation, we don't need to maintain two-pointers for the front and rear if we use a circular linked list. We can maintain a pointer to the last inserted node and the front can always be obtained as next of last.
2. Circular lists are useful in applications to go around the list repeatedly. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.

[DSA Tutorial](#) [Array](#) [Strings](#) [Linked List](#) [Stack](#) [Queue](#) [Tree](#) [Graph](#)

[Sign In](#)

- advanced data structures like the [B+ tree](#).
4. Circular linked lists can be used to implement circular queues, which are often used in operating systems for scheduling processes and managing memory allocation.
 5. Used in database systems to implement linked data structures, such as B+ trees, which are used to optimize the storage and retrieval of data.
 6. Circular linked lists can be used in networking. For instance, to implement circular buffers for streaming data, such as video and audio, in networking applications.
 7. Video games use circular linked lists to manage sprite animations. Each frame of the animation is represented as a node in the list, and the last frame is connected to the first frame to create a loop.
 8. Circular linked lists can be used to represent a buffer of audio or signal data in signal processing applications. The last node is connected to the first node to create a loop, and the processing algorithms can efficiently iterate over the data.
 9. Traffic light control systems use circular linked lists to manage the traffic light cycles. Each phase of the traffic light cycle is represented as a node in the list, and the last node is connected to the first node to create a loop.

Application of Doubly Linked Lists:

1. Redo and undo functionality.
2. Use of the Back and forward button in a browser.
3. The most recently used section is represented by the Doubly Linked list.
4. Other Data structures like Stack, Hash Table, and Binary Tree can also be applied by Doubly Linked List.
5. Used to implement game objects and their interactions in a game engine.
6. Used in networking.
7. Used in Graph algorithms.
8. Operating systems use doubly linked lists to manage the process scheduling. Each process waiting to be executed is represented as a node in the doubly linked list, and the operating system can easily traverse the list in both directions to manage the process queue.

Example:

Design a data structure that supports following operations efficiently.

1. *getMin : Gets minimum*
2. *extractMin : Removes minimum*
3. *getMax : Gets maximum*
4. *extractMax : Removes maximum*
5. *insert : Inserts an item. It may be assumed that the inserted item is always greater than maximum so far. For example, a valid insertion order is 10, 12, 13, 20, 50.*

Explanation: *Doubly linked list is the best solution here. We maintain head and tail pointers, since inserted item is always greatest, we insert at tail. Deleting an item from head or tail can be done in $O(1)$ time. So all operations take $O(1)$ time.*

Recent Articles on Linked List

[Comment](#)[More info](#)

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem

**Corporate & Communications Address:**

A-143, 7th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar Pradesh
(201305)

Registered Address:

K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305

**Company**

About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
GATE
Software and Tools

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Explore

POTD
Job-A-Thon
Community
Blogs
Nation Skill Up

Courses

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Preparation Corner

Aptitude
Puzzles
GfG 160
DSA 360
System Design